Reward Scoring and Data Provenance in the ARC Disaster-Response Environment

ARC_Game RL — technical reference

June 23, 2026

Abstract

This document specifies, at the level of individual code paths, (i) where every quantity used to compute the reward originates inside the Unity game server and how it reaches the Python scorer, and (ii) how each reward sub-component is computed. The guiding architectural principle is a strict separation of concerns: **Unity reports only raw cumulative facts; all scoring, weighting, and clamping live in Python** so the reward can be retuned without a Unity rebuild. The rules-based reference policy is documented separately in a companion note.

Contents

1 Architecture: Unity Reports Facts, Python Scores

The environment is a Gym wrapper (`arc_game_gym_env_tcp.py`) that drives a headless Unity build over a TCP socket. On each `step()` the wrapper executes the agent's actions, sends an `advance_time` request, and receives the post-advance game state as a JSON payload. All reward-relevant quantities are carried in a single sub-object of that payload, `rewardMetrics`, populated by the Unity component `RewardMetricsTracker`. Unity performs *no* scoring: it only accumulates counts. The pipeline is shown in Figure 1.

2 The Raw Metrics Payload

Every field in `rewardMetrics` is a *cumulative-to-date* quantity, assembled by `BuildPayload()` on the `RewardMetricsTracker` and attached to the outgoing state in `TaskSystem.GetCurrentGameState` (which sets `state.rewardMetrics` to the payload). Table 1 lists each field, the Unity source that updates it, and the triggering game event.

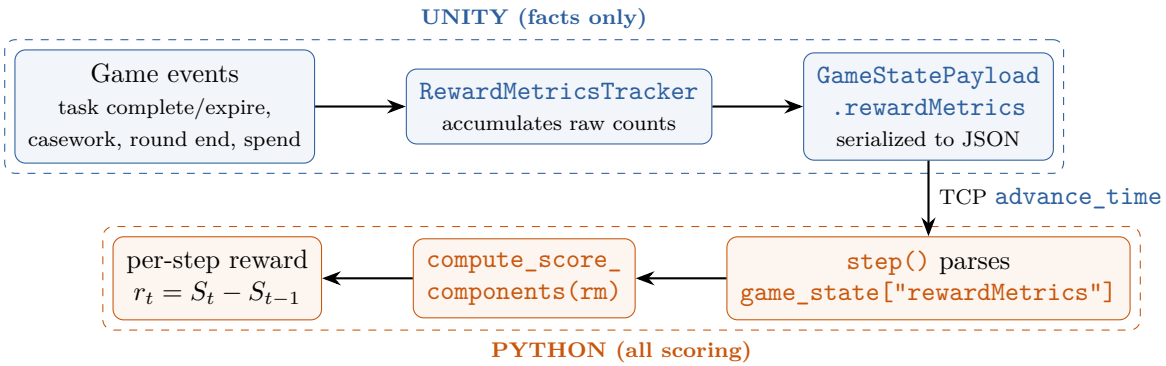

Figure 1: Reward pipeline. Unity accumulates raw cumulative quantities and ships them in `rewardMetrics`; Python computes the composite score and the per-step reward as its round-over-round delta.

Two design choices worth highlighting.

- **People-based fulfillment.** Food/Lodging success is measured in *people*, not tasks: a task that "completes" but delivers nobody (e.g. an immediate evacuation with no valid destination) adds to `*Resolved` but not `*Fulfilled`. This makes the satisfaction ratio track actual service delivered.
- **Worker allocation as person-rounds.** Worker use is sampled once per round and summed, so `cumWorkingWorkers` etc. measure occupancy integrated over time, which the scorer normalizes by the available worker-capacity `daysCompleted` × `totalWorkers`.

3 Score Components (Python)

All formulas below are implemented in `compute_score_components` (`arc_game_gym_env_tcp.py`). Let rm denote the `rewardMetrics` dict. Two helpers are used throughout:

$$\mathrm{clip}_{[0,1]}(x) = \min\big(1, \max(0, x)\big), \qquad \mathrm{ratio}(n, d) = \begin{cases} n/d & d \neq 0 \\ 0 & d = 0. \end{cases} \tag{1}$$

3.1 Satisfaction (higher is better)

Each satisfaction term is a clamped needs-met ratio scaled by a unit weight ($w_{\text{food}} = w_{\text{lodging}} = w_{\text{workeruse}} = w_{\text{casework}} = 1$):

$$\text{sat\_food} = \mathrm{clip}_{[0,1]}\Big(\mathrm{ratio}(\texttt{foodFulfilled}, \texttt{foodResolved})\Big)\, w_{\text{food}}, \tag{2}$$

$$\text{sat\_lodging} = \mathrm{clip}_{[0,1]}\Big(\mathrm{ratio}(\texttt{lodgingFulfilled}, \texttt{lodgingResolved})\Big)\, w_{\text{lodging}}, \tag{3}$$

$$\text{casework\_sat} = \mathrm{clip}_{[0,1]}\Big(\mathrm{ratio}(\texttt{caseworkProcessed}, \texttt{caseworkRequested})\Big)\, w_{\text{casework}}. \tag{4}$$

Field	Unity source (call site)	Updated when / semantics
foodResolved, foodFulfilled	RecordTaskResolution (TaskSystem.cs 1100 / 1199 / 1276); AddLateDelivery (614)	A Food-tagged Demand/Emergency task resolves. *People-based*: resolved += demandQuantity; fulfilled += min(delivered,demand). Late deliveries credit fulfillment retroactively (capped at resolved).
lodgingResolved, lodgingFulfilled	RecordTaskResolution / AddLateDelivery (same paths)	As above for Lodging-tagged tasks (relocation / housing).
caseworkRequested	RecordCaseworkRequested (ClientStayTracker.cs 320)	A "Casework Request" is generated for a client group housed too long; adds group.clientCount people.
caseworkProcessed	RecordCaseworkProcessed (ClientStayTracker.cs 205)	Clients are removed (sent home) via a staffed casework site; adds the number removed.
cumWorkingWorkers, cumTrainingWorkers, cumIdleWorkers	OnRoundEnded (GlobalClock.cs 458) snapshots GetWorkerStatistics	At each round end, the working / in-training / free worker counts are summed in, giving cumulative *person-rounds*.
roundsCompleted	OnRoundEnded	Count of simulated rounds elapsed.
daysCompleted	GlobalClock .GetCurrentDay()	Current in-game day (worker-capacity denominator).
totalWorkers	GetWorkerStatistics (present workers)	Snapshot of all workers present (working + training + free).
foodSpend, lodgingSpend, workerSpend, caseworkSpend	SatisfactionAndBudget .Cumulative*Spend (tallied in RemoveBudget by SpendCategory)	Dollars spent per category. lodgingSpend absorbs the recurring motel charge: MotelCostManager bills residents × \$200/day via RemoveBudget(...,Lodging).

Table 1: The rewardMetrics payload: every field, its originating Unity system, and the game event that updates it. All quantities are cumulative over the episode.

The worker-use term blends the three allocation states (utilization counts fully, training half, idle zero) and normalizes by total worker-capacity:

$$\text{capacity} = \max(\texttt{daysCompleted}, 1) \cdot \max(\texttt{totalWorkers}, 1), \tag{5}$$

$$\text{sat\_worker\_use} = \text{clip}_{[0,1]}\left(\frac{w_{\text{work}}\,\texttt{cumWorking} + w_{\text{train}}\,\texttt{cumTraining} + w_{\text{idle}}\,\texttt{cumIdle}}{\text{capacity}}\right) w_{\text{workeruse}}, \tag{6}$$

with $(w_{\text{work}}, w_{\text{train}}, w_{\text{idle}}) = (1,\ 0.5,\ 0)$. The total satisfaction is the sum:

$$\boxed{\text{satisfaction} = \text{sat\_food} + \text{sat\_lodging} + \text{sat\_worker\_use} + \text{casework\_sat}} \tag{7}$$

3.2 Cost-Efficiency (lower is better; subtracted)

Each cost term is dollars-spent per unit-of-service, scaled by a small weight and *capped* (not clamped) at $\tau = 1$. Spending with zero service is maximally inefficient and hits the cap:

$$\text{cost}(\text{spend},\ \text{service},\ w) = \min\left(\frac{\text{spend}}{\max(\text{service}, 1)}\, w,\ \tau\right), \qquad \tau = 1. \tag{8}$$

$$\text{cost\_food} = \text{cost}(\texttt{foodSpend},\ \texttt{foodFulfilled},\ w_c), \tag{9}$$
$$\text{cost\_lodging} = \text{cost}(\texttt{lodgingSpend},\ \texttt{lodgingFulfilled},\ w_c), \tag{10}$$
$$\text{cost\_worker} = \text{cost}(\texttt{workerSpend},\ \texttt{cumWorking},\ w_c), \tag{11}$$
$$\text{cost\_casework} = \text{cost}(\texttt{caseworkSpend},\ \texttt{caseworkProcessed},\ w_c), \tag{12}$$

with $w_c = 0.0002$ for all four categories (so \$5000/service saturates the cap). The aggregate is

$$\text{cost\_efficiency} = \text{cost\_food} + \text{cost\_lodging} + \text{cost\_worker} + \text{cost\_casework}. \tag{13}$$

3.3 Composite Score and Per-Step Reward

$$\boxed{S = \text{satisfaction} - \text{cost\_efficiency}} \tag{14}$$

S is a cumulative-to-date quantity. The Gym reward at round t is its round-over-round delta, which telescopes so the undiscounted return equals the final score:

$$r_t = S_t - S_{t-1}, \qquad \sum_{t=1}^{T} r_t = S_T - S_0. \tag{15}$$

Weight	**Value**	**Weight**	**Value**
$w_{\text{food}}, w_{\text{lodging}}, w_{\text{workeruse}}, w_{\text{casework}}$	1.0	w_c (all cost categories)	0.0002
w_{work} (working)	1.0	cost cap τ	1.0
w_{train} (in-training)	0.5	w_{idle} (idle)	0.0

Table 2: Reward weights (`REWARD_WEIGHTS`). All scoring constants live in Python and are retunable without rebuilding Unity.

Reading the score. Satisfaction has a natural ceiling of 4.0 (four unit-weighted terms each ≤ 1); cost-efficiency can subtract up to 4.0 (four capped terms). In practice strong policies land near $S \approx$ 2–2.5: high needs-met ratios minus modest per-service cost. Termination occurs if in-game satisfaction drops to 0; episodes otherwise truncate at the round limit.